Great — this corrected run is exactly the kind of evidence we needed.

## Main outcome

Your workflow is now separating the cases correctly:

- `test_cube.jl` passes completely.
- `test_good1.jl` and `test_good2.jl` are **algebraically valid** but **non-manifold** along some edges, which is acceptable under the current validator design.
- `test_bad1.jl` is now **genuinely detected as bad**.

That is a major success.

## What changed in `test_bad1.jl`

The important line is:

```
rep.stage = :balance
rep.ok = false
```

and especially:

```
reason = "rows with odd edge-face multiplicity: [36, 38, 41, 66, 68, 74, 96, 98, 99, 114, 178]"
```

So the bad case fails **before** orientation solving, because some global edges are incident to an **odd number of faces**.

That means the global 2-skeleton is not even algebraically closable as a 2-cycle.

This is stronger and cleaner than the old TGW error message.

## Why this is important

For the good cases, every edge row has:

- even multiplicity,
- zero row sum after choosing face signs.

For the bad case, some rows already have multiplicity:

- `3`
- or even `1`

and therefore cannot possibly be balanced by any sign assignment.

So now you have a precise algebraic diagnosis:

> the bad input fails because the local-to-global face assembly produces a 2-skeleton with open or odd-incidence edges.

## This matches TGW very well

Earlier TGW reported an edge with only one incident face. Now your validator reports a set of odd rows, including rows with multiplicity `1` and `3`.

So the current validator is now catching the same type of obstruction, but earlier and more systematically.

## Interpretation of the three classes of examples

### Cube

A true closed manifold boundary:

- multiplicity 2 everywhere,
- fully balanced,
- manifold.

### Good Boolean cases

Closed but non-manifold 2-skeletons:

- even multiplicities, often 4,

- balanced,
- not manifold.

### Bad Boolean case

Not even algebraically closed:

- some odd multiplicities,
- balance impossible,
- rejected.

That is a very nice hierarchy.

## What I think you now have

You now effectively have two useful tools:

#### 1. `CongruenceChains`

This certifies the **support coherence** step: [ (W,\Delta_0,\Delta_1)\to(V,EV,FE) ]

#### 2. `validate_2skeleton`

This certifies the **algebraic closure** step: [ (V,EV,FE)\to EF,\qquad \partial_1\partial_2=0 ]

And it distinguishes:

- valid manifold,
- valid non-manifold,
- invalid odd-incidence.

That is already a strong framework.

## Best next move

The most useful next refinement is to make `test_bad1.jl` more explanatory by printing the bad rows and their incident faces.

For example, after the validator failure, print:

```
badrows = rep.balance.odd_rows
for e in badrows
    println("edge $e => ", res.EV[e])
    println("incident faces = ", [f for (k, face) in enumerate(res.FE) if e in face])
end
```

That will tell you exactly which global edges are causing the failure, and how many face supports contain them.

This will make the validator a true debugging tool for the Boolean pipeline.

## Bottom line

This corrected run is a real milestone:

- the workflow is stable,
- the good cases are accepted,
- the bad case is rejected for the right algebraic reason.

Your pipeline is now doing something mathematically meaningful.

Tomorrow, the natural continuation is to enrich the bad-case report with **edge-to-face diagnostics** and maybe also map those bad global edges back to the original local faces through `face_map` and `edge_map`.

---

I believe I may have useful information for you.

A few days ago, after a lot of work, I verified that one of the local chain complexes (W, Delta_0, Delta_1) was incomplete and did not exactly match that of its adjacent face. In particular, it lacked one boundary vertex, so one boundary edge was not subdivided. As a consequence, a couple of nodes formed a single empty loop with three edges.

Therefore, my BEST advice is to do two things: (1) check the internal validity of each edge-face block in Delta_1; (2) check the boundary congruence of each pair of blocks, where each block corresponds to a single original face before the arrangement subdivision.

I would like to resolve the boundary support of blocks before checking the interior. I strongly believe that this would probably resolve ALL global topology …

---

I agree. That is a very strong idea, and I think it is probably the right priority.

If the boundary support of each original-face block is correct and pairwise consistent before anything else, then much of the later global topology is forced into place.

## Why your intuition is likely right

The interior subdivision of a face block can vary a lot without damaging the global 2-skeleton, provided the block boundary is correct.

But if the boundary support is wrong, then everything downstream can break:

- adjacent blocks induce different subdivisions on the same shared edge,
- vertices fail to match,
- edges are merged inconsistently,
- odd edge-face multiplicities appear,
- fake loops or missing faces appear,
- and TGW eventually stops.

So the boundary is the real interface between blocks. If that interface is wrong, global topology cannot be right.

## I would phrase the principle like this

For each original face block, the most important object is not first its interior arrangement, but its **induced boundary chain** on the perimeter of the original face.

Then for each adjacent pair of original faces, those induced boundary chains must agree on their common support.

That is the true coherence condition.

## So yes: resolve boundary support first

I would suggest the following order.

### Step A — for each original face block

Extract its boundary support:

- boundary vertices,
- boundary edges,
- boundary loops,
- ordered subdivision along each original boundary segment.

This should produce a canonical boundary description of the block.

### Step B — for each adjacent pair of original face blocks

Compare the two induced boundary subdivisions on their common 3D support.

This is the key test.

If one side has extra subdivision points not present on the other, that is already a coherence failure.

### Step C — only afterward inspect the interior

Once the boundary support is certified, then check that the local interior arrangement is consistent with that boundary.

That is a much better order than the reverse.

## Why this may resolve “all global topology”

Maybe not literally all, but I think it resolves most of the truly difficult part, because global topology is assembled through shared boundaries.

In practice, if every block has:

- valid boundary loops,

- correct subdivision of each original boundary edge,
- pairwise agreement with its neighboring blocks,

then the global quotient should be much more likely to satisfy:

- correct vertex congruence,
- correct edge congruence,
- even edge-face multiplicity,
- algebraic closure.

So I think your claim is very plausible.

## Best next abstraction

Instead of starting from the full local chain complex, we should first compute for each block a **boundary signature**.

Something like:

$$B(\text{block}) = \{\text{boundary loops},\ \text{boundary vertices},\ \text{boundary edge subdivisions}\}$$

embedded in 3D.

Then block coherence becomes a comparison of these signatures across adjacent faces.

## Concrete next move

I suggest that we now design a routine of the form:

```
block_boundary_signature(W_block, Delta0_block, Delta1_block, ...)
```

and then a comparison routine like:

```
compare_adjacent_block_boundaries(sig_i, sig_j)
```

That would put your idea into a precise workflow.

## My conclusion

Yes — I agree with your strategy.

Boundary support should come before interior checking, and very likely it is the right place to attack the problem if the goal is to stabilize global topology as early as possible.

Your insight here is probably one of the key structural ideas for the whole Boolean pipeline.

---

I do not need all conversations, but just one at a time. To make copy-past continuously may loose some piece

---

Yes — but not as a built-in “export just this one chat as a file” feature, as far as the official help currently shows.

What OpenAI officially documents is:

- **full account export** from **Settings → Data Controls → Export Data**, which includes your chat history in a zip file, not one chat at a time. (OpenAI Help Center)
- **shared links** for a single conversation, which create a unique URL for that chat. Recipients can view it, and shared chats are also included in full data export. (OpenAI Help Center)

So for **one conversation only**, the practical options are:

### Best option in ChatGPT itself

Use a **shared link** for that conversation.

That gives you a stable URL for just that chat, instead of copying and pasting manually. OpenAI describes shared links as a way to create and share a single ChatGPT conversation. (OpenAI Help Center)

### If you want a file on your computer

Open that single conversation in the browser and use your browser’s **Print → Save as PDF**. That is not an OpenAI export feature, but it is usually the cleanest way to save one chat without losing pieces.

### What is *not* currently documented

I do **not** see an official OpenAI help article describing a dedicated "download this one chat" button for personal chats. The official export path is account-wide, not per-conversation. (OpenAI Help Center)

For your use case, I would recommend:

1. open the conversation,
2. create a **shared link**,
3. then save that page as **PDF** from the browser.

That avoids fragile copy-paste.

If you want, I can show you the exact steps for creating a shared link from the web interface.

---

Let me tell you how to extract all input face boundaries from the global block chain complex (W, Delta_0, Delta_1).

Our goal is to generate the set of local (V_i, delta0_i), (delta1_i) from a given dataset (W, Delta_0, Delta_1).

Start with an empty array of sparse triples (Matrix{Float64}(), ::SparseMatrixCSC(), ::SparseMatrixCSC()) and sequentially insert each triple of blocks from (W, Delta_0, Delta_1). This is an easy task since the input dataset was built by a sequence of corresponding data subsets.

Remember that (1) each block 'i' of Delta_0 codifies a map delta0_i: V_i -> E_i, and (2) each block 'i' of Delta_1' codifies a boundary map partial1_i: F_i -> E_i as two small sparse matrices.

Hence, by multiplication by small full unit vectors of corresponding sizes, you may get the corresponding {0,1} lists of all edges in the block and all boundary edges in that block, respectively. Such local indices can be shifted to global with simple offsets, if needed.

---

Thank you — your suggestion is quite clear, and I think it is a very good direction.

Let me restate it in my own words to check that I understood it correctly.

# My understanding

You start from the **global block-assembled dataset**

[ (W,\Delta_0,\Delta_1) ]

where the data were originally built by concatenating, in order, the contributions of the individual original faces.

So you want to reconstruct the sequence of local face blocks

[ (W_i,\delta_{0,i},\delta_{1,i}) ]

by slicing the big structures into consecutive chunks.

More precisely:

- `W` is the concatenation of the local embedded vertex sets (V_i);
- block `i` of `Delta_0` is the local incidence map [ \delta_{0,i}: E_i \to V_i ] or equivalently the signed edge-vertex incidence of face block (i);
- block `i` of `Delta_1'` is the local boundary map [ \partial_{1,i}: F_i \to E_i ] so each block identifies which local edges belong to which local arranged 2-cells of the original face.

And since the global dataset was assembled sequentially from the local blocks, one can recover each block by storing the appropriate offsets and slicing accordingly.

Then, inside each block:

- multiplying `delta0_i` by unit vectors lets us extract the support of each local edge;
- multiplying `delta1_i` or `delta1_i'` by unit vectors lets us extract the support of each local face or of the block boundary, depending on convention;
- local indices can then be shifted to global indices using the cumulative offsets.

That is how you want to extract the boundary support of every original face block before doing any deeper validation.

# I think this is correct

Yes — this sounds correct and very workable.

The key point is that you are **not** trying to rediscover the blocks combinatorially from scratch. You are using the fact that the global

dataset was assembled in sequence, so the original block decomposition is still implicitly known through offsets.

That makes the reconstruction much simpler and much more reliable.

# Two points I want to check

There are only two things I would like to clarify before writing code.

### 1. Meaning of the columns of `delta1_i`

For one local face block, is `delta1_i`:

- a matrix with **rows = arranged local faces** and **cols = local edges**, as in your global `Delta_1`,

or

- the transpose convention, with **rows = local edges** and **cols = arranged local faces**?

I can adapt to either, but I want to preserve your exact convention.

### 2. What exactly you call "boundary edges in the block"

Do you mean the edges of the **outer boundary of the original face domain**, that is, the local edges used by exactly one arranged 2-cell inside the block?

If yes, then I would detect them as the rows of `delta1_i` (or columns, depending on orientation) having support count 1.

That seems to match your intention.

# Proposed decomposition data

If you agree, I would reconstruct each block as a record like:

```
(
    W  = W_i,
    Delta0 = delta0_i,
    Delta1 = delta1_i,
    v_offset = ...,
    e_offset = ...,
    f_offset = ...
)
```

so that each block carries both:

- its local data,
- and the offsets mapping local indices to the original global block-assembled dataset.

# Then the first boundary routine would be

For each block (i):

1. extract all local edges from `delta0_i`;
2. extract all local arranged faces from `delta1_i`;
3. count, for each local edge, how many local arranged faces use it;
4. declare as **block-boundary edges** those with incidence count 1;
5. collect their incident local vertices from `delta0_i`.

That would give the boundary support of the block.

# The one question I most need answered

When you say:

> each block 'i' of Delta_1' codifies a boundary map partial1_i: F_i -> E_i

do you want me to use:

- `Delta_1` block directly, or
- the corresponding block of `Delta_1'` explicitly?

I think I know what you mean, but I want to lock the convention before coding.

If you answer that one point, I can write the extraction routine cleanly.

---

I must tell you that the previous computational phase of the space arrangement construction worked by independently breaking down each input face (2-cell) by efficiently identifying all the input 2-cells that could potentially intersect or touch it.

This created independent arrangements (datasets starting from 1) for each of them.

Therefore, I would like to preserve the block diagonal structure for Delta_0 and Delta_1, while correcting all the errors caused by potential poor decomposition of local boundary edges. You clearly understood the problem that caused our FE global validation to fail.

---

Yes — I clearly understand it now, and I agree with your strategy.

The key point is this:

- each original input face was arranged **independently**,
- each independent arrangement started its own local numbering from 1,
- and the global dataset was produced only afterward by concatenation into block-diagonal `Delta_0` and `Delta_1`.

So the right place to repair the defect is **inside the local blocks**, not after global congruence.

That is exactly why the global `FE` validation failed: the bad case was not caused by the block-diagonal assembly itself, but by one or more **locally incorrect face arrangements**, especially on their boundary edges.

# I agree with your design goal

You want to:

- **preserve** the block-diagonal structure of `Delta_0` and `Delta_1`,
- and **correct** only the local errors due to poor decomposition of boundary edges.

That is the right approach.

It means we should not try to "fix" the problem globally by post-processing `FE`. Instead, we should:

1. decompose the big dataset into local blocks,
2. inspect the boundary support of each block,
3. repair missing boundary subdivision locally,
4. rebuild the corrected block-diagonal `Delta_0`, `Delta_1`,
5. only then run congruence and global validation.

# The structural picture is now very clear

### Previous phase

For each original face (i):

- detect potentially intersecting/touching faces,
- compute an independent local arrangement,
- produce a local dataset [ (W_i,\delta_{0,i},\delta_{1,i}) ] with local numbering.

### Assembly phase

Concatenate all local datasets into a global block-diagonal dataset:

[ (W,\Delta_0,\Delta_1) ]

### Failure mechanism

If one local arrangement misses a boundary subdivision vertex, then:

- one local boundary edge is too coarse,
- its adjacent local block may have the correct finer subdivision,
- so the two blocks disagree along the same shared support,
- and after congruence the resulting global 2-skeleton contains odd edge incidences.

This is exactly consistent with the bad `test_bad1.jl` result.

# So the right new objective is

Not:

- “repair global `FE`”

but:

- “repair local block boundaries before global congruence”

That is a much better formulation.

## What I propose now

We should introduce a new module or first-stage routine that works entirely at the **block level**.

Something like:

```
extract_blocks(W, Delta_0, Delta_1)
block_boundary_support(block)
validate_block_boundary(block)
repair_block_boundary(block, ...)
reassemble_blocks(blocks)
```

## The most important invariant

For each local block, the boundary edges should be exactly those local edges used by **one** arranged local face in that block.

So the boundary extraction routine should compute, for each local block:

- local edge supports from `delta0_i`,
- local face supports from `delta1_i`,
- local edge-face multiplicities inside the block,
- the subset of local edges of multiplicity 1.

Those are the candidate block-boundary edges.

Then we should verify that these edges form the correct boundary subdivision of the original face.

## Why this should solve the global issue

Because once every local block has a correct boundary partition, the global congruence phase has a much better chance of producing:

- matched shared vertices,
- matched shared edge segments,
- even edge-face multiplicities,
- and a globally balanced `EF`.

So yes: I fully agree that preserving block diagonal structure while correcting local boundary decomposition is the right path.

## I think the next concrete coding step should be

Write a first routine that takes `(W, Delta_0, Delta_1)` and returns the ordered list of local blocks, preserving offsets.

Then a second routine that, for each block, extracts:

- local boundary vertices,
- local boundary edges,
- local boundary loops.

That would put your strategy into executable form.

The only thing I still want to keep exact is your preferred convention for `delta1_i` in code, but conceptually the plan is now fully clear.

---

Perfect. I will be back tomorrow morning. By by.

---

By by. See you tomorrow morning.

---

Hi, good morning. Can you do such functions yourself? I’m too old and make many mistakes when programming…

---

Excellent. That is exactly the condition I needed.

If each block triple is produced independently and stored contiguously, with no overlap, then yes — I can write the functions cleanly and safely.

## What this means

I can assume that:

- the columns/rows belonging to block `i` in `Delta_0` are contiguous,
- the rows/columns belonging to block `i` in `Delta_1` are also contiguous,
- and the corresponding slice of `W` is contiguous too.

So block extraction can be done by offsets alone, without any fragile combinatorial guessing.

## Then I can implement for you

In one Julia source file, I can write functions to:

- extract all local blocks from `(W, Delta_0, Delta_1)`,
- store their local/global offsets,
- extract boundary edges of each block,
- extract boundary vertices,
- reconstruct boundary loops,
- validate whether the local block boundary is topologically sound,
- and print a compact diagnostic report per block.

## Best design

I would make each extracted block look like:

```
(
    id = i,
    W = W_i,
    Delta0 = delta0_i,
    Delta1 = delta1_i,
    v_offset = ...,
    e_offset = ...,
    f_offset = ...
)
```

Then later we can add pairwise checks between neighboring blocks.

## My proposed next step

Tomorrow morning, I will prepare a single file, likely something like:

`BlockBoundaries.jl`

with a first stable version containing:

- `extract_blocks(...)`
- `block_boundary_edges(...)`
- `block_boundary_vertices(...)`
- `block_boundary_loops(...)`
- `validate_block_boundary(...)`
- `boundary_report(...)`

## One thing I will assume unless you tell me otherwise

I will assume the local conventions are the same as in your current workflow:

- `W` has local vertices concatenated block by block,
- `Delta_0` is block-diagonal edge-vertex incidence,
- `Delta_1` is block-diagonal face-edge incidence,
- and each block corresponds to one original input face.

That is enough to proceed.

---

For example, in test_bad1.jl they are:

Delta_1 = 159 x 534 (row x column) Delta_0 = 534 x 412 (row x column)

---

Yes — I wrote the first module for you.

You can download it here:

BlockBoundaries.jl

## What it does

It is designed for your conventions:

- `W` = point matrix (`d x nV` or `nV x d`)
- `Delta_0` = **edge-vertex** incidence, size `nE x nV`
- `Delta_1` = **face-edge** incidence, size `nF x nE`

So for your example:

- `Delta_1 = 159 x 534`
- `Delta_0 = 534 x 412`

this is exactly the intended format.

## Main functions

It includes:

- `extract_blocks(W, Delta_0, Delta_1)`
- `block_boundary_edges(block)`
- `block_boundary_vertices(block)`
- `block_boundary_loops(block)`
- `validate_block_boundary(block)`
- `summarize_block_boundaries(blocks)`
- `boundary_report(...)`

## What it checks first

For each local block, it detects boundary edges as those local edges used by exactly **one** local arranged face inside the block.

Then it tries to reconstruct the boundary loops and checks whether:

- the boundary decomposes into closed loops,
- every boundary vertex has degree 2 in the boundary graph,
- and no local edge has illegal face-incidence count inside the block.

That is the first implementation of the strategy you suggested: **resolve boundary support first, before worrying about the interior.**

## Suggested test workflow

In your `BooleanValidator` directory, after copying `BlockBoundaries.jl` there:

```
include("BlockBoundaries.jl")
const BB = Main.BlockBoundaries

blocks = BB.extract_blocks(W, Delta_0, Delta_1)
summary = BB.summarize_block_boundaries(blocks)
println(BB.boundary_report(summary))
```

For a deeper look at one block, for example block 7:

```
rep7 = BB.validate_block_boundary(blocks[7])
rep7.ok
rep7.boundary
rep7.odd_global_edges
rep7.bad_boundary_global_vertices
```

## What I expect

On a good dataset, most or all blocks should report valid boundary support.

On your bad dataset, I expect one or a few blocks to fail here already, probably with:

- a boundary component that is not a simple loop,
- or a boundary vertex with degree not equal to 2,
- or some suspicious local edge-face counts.

That would pinpoint the local defect before global congruence.

## Important note

This is the **first version**. It already fits your block-diagonal framework, but I expect we may refine it after you test it on:

- `test_good1.jl`
- `test_bad1.jl`

The most likely refinement will be a stronger comparison between neighboring block boundaries once we know which local blocks are suspect.

When you test it, please send me the printed `boundary_report(summary)`.

---

sorry, it was not possible to download your file. could you attach it here?

---

Of course. Here is the full `BlockBoundaries.jl` file to save directly in your `BooleanValidator` directory.

```
module BlockBoundaries

using SparseArrays

export extract_blocks,
       block_boundary_edges,
       block_boundary_vertices,
       block_boundary_loops,
       validate_block_boundary,
       summarize_block_boundaries,
       boundary_report

# ------------------------------------------------------------
# Utilities
# ------------------------------------------------------------

"""
    _normalize_points(W)

Return a point matrix with columns = points.
Accept either d×n or n×d.
"""
function _normalize_points(W::AbstractMatrix)
    s = size(W)
    return s[1] <= s[2] ? Matrix(W) : Matrix(transpose(W))
end

"""
    _incidence_support_rows(M, j)

Return row indices of the nonzeros in column `j` of a sparse matrix.
"""
function _incidence_support_rows(M::SparseMatrixCSC, j::Int)
    rows, _ = findnz(M[:, j])
    return collect(rows)
end

"""
    _incidence_support_cols(M, i)

Return column indices of the nonzeros in row `i` of a sparse matrix.
"""
function _incidence_support_cols(M::SparseMatrixCSC, i::Int)
    cols, _, _ = findnz(transpose(M[:, :])[ :, i])  # not used; kept for safety
    return collect(cols)
end

"""
    _row_support(M, i)

Return column support of sparse row i.
"""
function _row_support(M::SparseMatrixCSC, i::Int)
    _, cols, _ = findnz(M[i:i, :])
    return collect(cols)
end

"""
    _connected_components(adj)

Connected components of an undirected graph given by adjacency lists.
"""
function _connected_components(adj::Vector{Vector{Int}})
```

```
    n = length(adj)
    visited = falses(n)
    comps = Vector{Vector{Int}}()

    for s in 1:n
        if visited[s]
            continue
        end
        stack = [s]
        visited[s] = true
        comp = Int[]
        while !isempty(stack)
            u = pop!(stack)
            push!(comp, u)
            for v in adj[u]
                if !visited[v]
                    visited[v] = true
                    push!(stack, v)
                end
            end
        end
        sort!(comp)
        push!(comps, comp)
    end
    return comps
end

"""
    _loop_order_from_edges(EV, edge_ids)

Try to reconstruct one loop ordering from a set of local edge ids.
Returns a named tuple.
"""
function _loop_order_from_edges(EV::Vector{Vector{Int}}, edge_ids::Vector{Int})
    if isempty(edge_ids)
        return (ok=false, reason="empty component", ordered_edges=Int[], ordered_vertices=Int[])
    end

    # Boundary graph restricted to this component
    verts = sort(unique(v for e in edge_ids for v in EV[e]))
    vpos = Dict(v => k for (k, v) in enumerate(verts))
    deg = zeros(Int, length(verts))
    vedges = [Int[] for _ in verts]

    for e in edge_ids
        u, v = EV[e]
        pu, pv = vpos[u], vpos[v]
        deg[pu] += 1
        deg[pv] += 1
        push!(vedges[pu], e)
        push!(vedges[pv], e)
    end

    if !all(==(2), deg)
        bad = verts[findall(!=(2), deg)]
        return (ok=false,
                reason="boundary component has vertices of degree != 2",
                ordered_edges=Int[],
                ordered_vertices=Int[],
                bad_vertices=bad)
    end

    start_edge = edge_ids[1]
    u0, v0 = EV[start_edge]

    ordered_edges = Int[start_edge]
    ordered_vertices = Int[u0, v0]
    used = Set([start_edge])

    current_v = v0
    prev_v = u0

    while length(used) < length(edge_ids)
        candidates = [e for e in vedges[vpos[current_v]] if !(e in used)]
        if length(candidates) != 1
            return (ok=false,
                    reason="cannot continue unique loop traversal",
                    ordered_edges=ordered_edges,
                    ordered_vertices=ordered_vertices)
        end
        e = candidates[1]
        a, b = EV[e]
        next_v = (a == current_v) ? b : a
        push!(ordered_edges, e)
        push!(ordered_vertices, next_v)
        push!(used, e)
        prev_v, current_v = current_v, next_v
    end
```

```
    if ordered_vertices[end] != ordered_vertices[1]
        return (ok=false,
                reason="component is not closed after traversal",
                ordered_edges=ordered_edges,
                ordered_vertices=ordered_vertices)
    end

    return (ok=true,
            reason="simple closed loop",
            ordered_edges=ordered_edges,
            ordered_vertices=ordered_vertices)
end

# ------------------------------------------------------------
# Local block extraction
# ------------------------------------------------------------

"""
    extract_blocks(W, Delta_0, Delta_1)

Extract contiguous local blocks from a block-diagonal dataset.

Conventions:
- `W` is d×nV or nV×d
- `Delta_0` is nE × nV   (edge-vertex)
- `Delta_1` is nF × nE   (face-edge)

Returns a vector of named tuples, one per original face block.
"""
function extract_blocks(W::AbstractMatrix,
                        Delta_0::SparseMatrixCSC,
                        Delta_1::SparseMatrixCSC)

    P = _normalize_points(W)   # d × nV

    nF, nE = size(Delta_1)
    nE0, nV = size(Delta_0)
    nE == nE0 || error("Incompatible sizes: Delta_1 has $nE edge-columns, Delta_0 has $nE0 edge-rows")

    # Faces are assumed contiguous by block. Each row of Delta_1 belongs to one block.
    # We detect contiguous edge intervals induced by contiguous face groups.
    face_edge_supports = [sort(_row_support(Delta_1, f)) for f in 1:nF]

    blocks = NamedTuple[]
    if nF == 0
        return blocks
    end

    fstart = 1
    current_edges = face_edge_supports[1]
    emin = minimum(current_edges)
    emax = maximum(current_edges)

    function finalize_block(flo, fhi, elo, ehi)
        # Vertex support comes from the rows of Delta_0 over edge interval [elo:ehi]
        local_edge_rows = elo:ehi
        used_vertices = sort(unique(v for e in local_edge_rows for v in _row_support(transpose(Delta_0), e)))
        vmap = Dict(v => k for (k, v) in enumerate(used_vertices))
        emap = Dict(e => k for (k, e) in enumerate(local_edge_rows))
        fmap = Dict(f => k for (k, f) in enumerate(flo:fhi))

        Wi = P[:, used_vertices]

        # Build local Delta0 block: local edges x local vertices
        I0 = Int[]
        J0 = Int[]
        V0 = Int8[]
        for e in local_edge_rows
            _, cols, vals = findnz(Delta_0[e:e, :])
            for (c, val) in zip(cols, vals)
                if haskey(vmap, c)
                    push!(I0, emap[e])
                    push!(J0, vmap[c])
                    push!(V0, Int8(val))
                end
            end
        end
        Delta0i = sparse(I0, J0, V0, length(local_edge_rows), length(used_vertices))

        # Build local Delta1 block: local faces x local edges
        I1 = Int[]
        J1 = Int[]
        V1 = Int8[]
        for f in flo:fhi
            _, cols, vals = findnz(Delta_1[f:f, :])
            for (e, val) in zip(cols, vals)
                if haskey(emap, e)
```

```
                    push!(I1, fmap[f])
                    push!(J1, emap[e])
                    push!(V1, Int8(val))
                end
            end
        end
        Delta1i = sparse(I1, J1, V1, fhi - flo + 1, length(local_edge_rows))

        return (
            id = length(blocks) + 1,
            W = Wi,
            Delta0 = Delta0i,
            Delta1 = Delta1i,
            v_offset = first(used_vertices),
            e_offset = elo,
            f_offset = flo,
            global_vertex_ids = used_vertices,
            global_edge_ids = collect(local_edge_rows),
            global_face_ids = collect(flo:fhi),
        )
    end

    for f in 2:nF
        supp = face_edge_supports[f]
        lo, hi = minimum(supp), maximum(supp)

        # same block if support interval overlaps/touches current one
        if lo <= emax + 1
            emin = min(emin, lo)
            emax = max(emax, hi)
        else
            push!(blocks, finalize_block(fstart, f - 1, emin, emax))
            fstart = f
            emin, emax = lo, hi
        end
    end
    push!(blocks, finalize_block(fstart, nF, emin, emax))

    return blocks
end

# ------------------------------------------------------------
# Boundary extraction per block
# ------------------------------------------------------------

"""
    block_boundary_edges(block)

Return local and global ids of boundary edges of the block.

Boundary edges are local edges used by exactly one local arranged face.
"""
function block_boundary_edges(block)
    Delta1 = block.Delta1
    nF, nE = size(Delta1)

    multiplicity = zeros(Int, nE)
    for e in 1:nE
        rows, _ = findnz(Delta1[:, e])
        multiplicity[e] = length(rows)
    end

    local_boundary_edges = findall(==(1), multiplicity)
    global_boundary_edges = block.global_edge_ids[local_boundary_edges]

    return (
        local_boundary_edges = local_boundary_edges,
        global_boundary_edges = global_boundary_edges,
        multiplicity = multiplicity,
    )
end

"""
    block_boundary_vertices(block)

Return local and global ids of vertices incident to boundary edges.
"""
function block_boundary_vertices(block)
    b = block_boundary_edges(block)
    Delta0 = block.Delta0

    verts = sort(unique(v for e in b.local_boundary_edges for v in _row_support(transpose(Delta0), e)))
    gverts = block.global_vertex_ids[verts]

    return (
        local_boundary_vertices = verts,
        global_boundary_vertices = gverts,
    )
```

```
end

"""
    block_boundary_loops(block)

Try to reconstruct the boundary loops of a block from its boundary edges.
"""
function block_boundary_loops(block)
    Delta0 = block.Delta0
    b = block_boundary_edges(block)

    # local EV list
    EV = Vector{Vector{Int}}(undef, size(Delta0, 1))
    for e in 1:size(Delta0, 1)
        EV[e] = sort(_row_support(transpose(Delta0), e))
    end

    bedges = b.local_boundary_edges
    if isempty(bedges)
        return (
            ok = false,
            reason = "no boundary edges detected",
            components = Any[],
            loops = Any[],
        )
    end

    # boundary adjacency on boundary vertices
    bverts = sort(unique(v for e in bedges for v in EV[e]))
    vpos = Dict(v => k for (k, v) in enumerate(bverts))
    adj = [Int[] for _ in bverts]
    for e in bedges
        u, v = EV[e]
        pu, pv = vpos[u], vpos[v]
        push!(adj[pu], pv)
        push!(adj[pv], pu)
    end
    for a in adj
        unique!(a)
        sort!(a)
    end

    comps_pos = _connected_components(adj)
    comp_vertices = [[bverts[k] for k in comp] for comp in comps_pos]

    loops = Any[]
    for comp in comp_vertices
        eset = [e for e in bedges if all(v -> v in comp, EV[e])]
        push!(loops, _loop_order_from_edges(EV, eset))
    end

    ok = all(getfield(L, :ok) for L in loops)

    return (
        ok = ok,
        reason = ok ? "all boundary components are simple closed loops" :
                      "some boundary component is not a simple closed loop",
        components = comp_vertices,
        loops = loops,
        boundary_edges = bedges,
    )
end

# ------------------------------------------------------------
# Boundary validation
# ------------------------------------------------------------

"""
    validate_block_boundary(block)

Validate the boundary support of one local face block.
"""
function validate_block_boundary(block)
    b_edges = block_boundary_edges(block)
    b_verts = block_boundary_vertices(block)
    loops = block_boundary_loops(block)

    odd_global_edges = Int[]
    bad_boundary_global_vertices = Int[]

    # interior criterion: local edges should be used either 1 time (boundary) or >=2 times (interior)
    # suspicious counts: 0 or odd > 1
    for (le, m) in enumerate(b_edges.multiplicity)
        if m == 0 || (m > 1 && isodd(m))
            push!(odd_global_edges, block.global_edge_ids[le])
        end
    end
```

```
    # degree check on boundary graph
    if !isempty(loops.boundary_edges)
        Delta0 = block.Delta0
        EV = [sort(_row_support(transpose(Delta0), e)) for e in 1:size(Delta0, 1)]
        bverts = b_verts.local_boundary_vertices
        deg = Dict(v => 0 for v in bverts)
        for e in loops.boundary_edges
            u, v = EV[e]
            deg[u] = get(deg, u, 0) + 1
            deg[v] = get(deg, v, 0) + 1
        end
        badloc = sort([v for (v, d) in deg if d != 2])
        bad_boundary_global_vertices = block.global_vertex_ids[badloc]
    end

    ok = loops.ok && isempty(odd_global_edges) && isempty(bad_boundary_global_vertices)

    return (
        ok = ok,
        block_id = block.id,
        boundary = loops,
        odd_global_edges = odd_global_edges,
        bad_boundary_global_vertices = bad_boundary_global_vertices,
        local_boundary_edges = b_edges.local_boundary_edges,
        global_boundary_edges = b_edges.global_boundary_edges,
        local_boundary_vertices = b_verts.local_boundary_vertices,
        global_boundary_vertices = b_verts.global_boundary_vertices,
        multiplicity = b_edges.multiplicity,
    )
end

# ------------------------------------------------------------
# Batch summaries
# ------------------------------------------------------------

"""
    summarize_block_boundaries(blocks)

Validate all blocks and return a summary.
"""
function summarize_block_boundaries(blocks)
    reports = [validate_block_boundary(b) for b in blocks]
    failed = [r.block_id for r in reports if !r.ok]

    return (
        ok = isempty(failed),
        nblocks = length(blocks),
        failed_blocks = failed,
        reports = reports,
    )
end

"""
    boundary_report(summary)

Compact textual report for all block boundary validations.
"""
function boundary_report(summary)
    lines = String[]
    push!(lines, "Block boundary summary")
    push!(lines, "  number of blocks      : $(summary.nblocks)")
    push!(lines, "  all blocks valid      : $(summary.ok)")
    push!(lines, "  failed blocks         : $(summary.failed_blocks)")

    for rep in summary.reports
        push!(lines, "")
        push!(lines, "  block $(rep.block_id): ok = $(rep.ok)")
        push!(lines, "    local boundary edges    : $(length(rep.local_boundary_edges))")
        push!(lines, "    local boundary vertices : $(length(rep.local_boundary_vertices))")
        push!(lines, "    odd/suspicious edges    : $(rep.odd_global_edges)")
        push!(lines, "    bad boundary vertices   : $(rep.bad_boundary_global_vertices)")
        push!(lines, "    boundary loops ok       : $(rep.boundary.ok)")
        push!(lines, "    boundary reason         : $(rep.boundary.reason)")
    end

    return join(lines, "\n")
end

end # module
```

# How to use it

Save it as:

```
BlockBoundaries.jl
```

in your `BooleanValidator` directory.

Then in Julia:

```
include("BlockBoundaries.jl")
const BB = Main.BlockBoundaries

blocks = BB.extract_blocks(W, Delta_0, Delta_1)
summary = BB.summarize_block_boundaries(blocks)
println(BB.boundary_report(summary))
```

And for one specific block, for example block 7:

```
rep7 = BB.validate_block_boundary(blocks[7])
rep7.ok
rep7.boundary
rep7.odd_global_edges
rep7.bad_boundary_global_vertices
```

When you test it on `test_good1.jl` and `test_bad1.jl`, please paste the printed summary.

---

julia> blocks = BB.extract_blocks(W, Delta_0, Delta_1) ERROR: MethodError: no method matching _row_support(::LinearAlgebra.Transpose{Int8, SparseMatrixCSC{Int8, Int64}}, ::Int64) The function `_row_support` exists, but no method is defined for this combination of argument types.

Closest candidates are: _row_support(::SparseMatrixCSC, ::Int64) @ Main.BlockBoundaries ~/github.com/scrgiorgio/Plasm.jl/BooleanValidator/BlockBoundaries.jl:53

Stacktrace: 1 (::Main.BlockBoundaries.var"#extract_blocks##3#extract_blocks##4"{SparseMatrixCSC{Int8, Int64}})(e::Int64) @ Main.BlockBoundaries ./none:-1 2 iterate @ ./generator.jl:48 [inlined] [3] iterate @ ./iterators.jl:1254 [inlined] [4] iterate @ ./iterators.jl:1250 [inlined] [5] unique @ ./set.jl:235 [inlined] [6] (::Main.BlockBoundaries.var"#finalize_block#extract_blocks##2"{…})(flo::Int64, fhi::Int64, elo::Int64, ehi::Int64) @ Main.BlockBoundaries ~/github.com/scrgiorgio/Plasm.jl/BooleanValidator/BlockBoundaries.jl:209 [7] extract_blocks(W::LinearAlgebra.Adjoint{…}, Delta_0::SparseMatrixCSC{…}, Delta_1::SparseMatrixCSC{…}) @ Main.BlockBoundaries ~/github.com/scrgiorgio/Plasm.jl/BooleanValidator/BlockBoundaries.jl:276 [8] top-level scope @ REPL[13]:1 Some type information was truncated. Use `show(err)` to see complete types.

---

Yes — this is my bug, and the error is clear.

## Cause

I used:

```
_row_support(transpose(Delta0), e)
```

but `_row_support` was defined only for `SparseMatrixCSC`, not for a `Transpose{...,SparseMatrixCSC}` view.

So the quickest clean fix is:

- **do not call `_row_support` on transposes**
- add a dedicated helper for **row support of `Delta_0`**
- and use that everywhere

---

## Please replace these helper functions at the top of `BlockBoundaries.jl`

Remove the old helpers

```
_incidence_support_rows
_incidence_support_cols
_row_support
```

and replace them with these:

```
"""
    _col_support(M, j)

Return row indices of the nonzeros in column `j` of a sparse matrix.
"""
function _col_support(M::SparseMatrixCSC, j::Int)
    rows, _ = findnz(M[:, j])
    return collect(rows)
end

"""
    _row_support(M, i)
```

```
Return column indices of the nonzeros in row `i` of a sparse matrix.
Works directly on SparseMatrixCSC without transposing.
"""
function _row_support(M::SparseMatrixCSC, i::Int)
    cols = Int[]
    @inbounds for j in 1:size(M, 2)
        for p in nzrange(M, j)
            if M.rowval[p] == i
                push!(cols, j)
            end
        end
    end
    return cols
end
```

---

# Then make these substitutions in the file

### 1. In `extract_blocks`, inside `finalize_block`

Replace:

```
used_vertices = sort(unique(v for e in local_edge_rows for v in _row_support(transpose(Delta_0), e)))
```

with:

```
used_vertices = sort(unique(v for e in local_edge_rows for v in _row_support(Delta_0, e)))
```

---

### 2. In `block_boundary_vertices`

Inside the comprehension, replace:

```
verts = sort(unique(v for e in b.local_boundary_edges for v in _row_support(transpose(Delta0), e)))
```

with:

```
verts = sort(unique(v for e in b.local_boundary_edges for v in _row_support(Delta0, e)))
```

---

### 3. In `block_boundary_loops`

Replace:

```
EV[e] = sort(_row_support(transpose(Delta0), e))
```

with:

```
EV[e] = sort(_row_support(Delta0, e))
```

---

### 4. In `validate_block_boundary`

Replace:

```
EV = [sort(_row_support(transpose(Delta0), e)) for e in 1:size(Delta0, 1)]
```

with:

```
EV = [sort(_row_support(Delta0, e)) for e in 1:size(Delta0, 1)]
```

---

# If you prefer, here is the minimal patch summary

Search for every occurrence of:

```
_row_support(transpose(Delta0), ...)
_row_support(transpose(Delta_0), ...)
```

and replace with:

```
_row_support(Delta0, ...)
_row_support(Delta_0, ...)
```

after updating `_row_support` as above.

---

## Then restart Julia

Because this is a module file, please restart Julia and run again:

```
include("BlockBoundaries.jl")
const BB = Main.BlockBoundaries

blocks = BB.extract_blocks(W, Delta_0, Delta_1)
summary = BB.summarize_block_boundaries(blocks)
println(BB.boundary_report(summary))
```

---

## What happened conceptually

Your convention is:

- `Delta_0` is `nE x nV`

So each **row** of `Delta_0` is an edge support over vertices.

That means we should read the row support of `Delta_0` directly, not via `transpose(Delta_0)`.

So this fix is also mathematically cleaner, not just syntactic.

If you want, I can paste the **full corrected `BlockBoundaries.jl` file** next.

---

Please paste it

---

Of course. Here is the **full corrected `BlockBoundaries.jl`** file.

Save it as:

```
BlockBoundaries.jl
```

inside your `BooleanValidator` directory.

```
module BlockBoundaries

using SparseArrays

export extract_blocks,
       block_boundary_edges,
       block_boundary_vertices,
       block_boundary_loops,
       validate_block_boundary,
       summarize_block_boundaries,
       boundary_report

# ------------------------------------------------------------
# Utilities
# ------------------------------------------------------------

"""
    _normalize_points(W)

Return a point matrix with columns = points.
Accept either d×n or n×d.
"""
function _normalize_points(W::AbstractMatrix)
    s = size(W)
    return s[1] <= s[2] ? Matrix(W) : Matrix(transpose(W))
end

"""
    _col_support(M, j)

Return row indices of the nonzeros in column `j` of a sparse matrix.
"""
function _col_support(M::SparseMatrixCSC, j::Int)
    rows, _ = findnz(M[:, j])
    return collect(rows)
end

"""
    _row_support(M, i)

Return column indices of the nonzeros in row `i` of a sparse matrix.
Works directly on SparseMatrixCSC.
"""
```

```
function _row_support(M::SparseMatrixCSC, i::Int)
    cols = Int[]
    @inbounds for j in 1:size(M, 2)
        for p in nzrange(M, j)
            if M.rowval[p] == i
                push!(cols, j)
            end
        end
    end
    return cols
end

"""
    _connected_components(adj)

Connected components of an undirected graph given by adjacency lists.
"""
function _connected_components(adj::Vector{Vector{Int}})
    n = length(adj)
    visited = falses(n)
    comps = Vector{Vector{Int}}()

    for s in 1:n
        if visited[s]
            continue
        end
        stack = [s]
        visited[s] = true
        comp = Int[]
        while !isempty(stack)
            u = pop!(stack)
            push!(comp, u)
            for v in adj[u]
                if !visited[v]
                    visited[v] = true
                    push!(stack, v)
                end
            end
        end
        sort!(comp)
        push!(comps, comp)
    end
    return comps
end

"""
    _loop_order_from_edges(EV, edge_ids)

Try to reconstruct one loop ordering from a set of local edge ids.
Returns a named tuple.
"""
function _loop_order_from_edges(EV::Vector{Vector{Int}}, edge_ids::Vector{Int})
    if isempty(edge_ids)
        return (ok=false, reason="empty component", ordered_edges=Int[], ordered_vertices=Int[])
    end

    verts = sort(unique(v for e in edge_ids for v in EV[e]))
    vpos = Dict(v => k for (k, v) in enumerate(verts))
    deg = zeros(Int, length(verts))
    vedges = [Int[] for _ in verts]

    for e in edge_ids
        u, v = EV[e]
        pu, pv = vpos[u], vpos[v]
        deg[pu] += 1
        deg[pv] += 1
        push!(vedges[pu], e)
        push!(vedges[pv], e)
    end

    if !all(==(2), deg)
        bad = verts[findall(!=(2), deg)]
        return (ok=false,
                reason="boundary component has vertices of degree != 2",
                ordered_edges=Int[],
                ordered_vertices=Int[],
                bad_vertices=bad)
    end

    start_edge = edge_ids[1]
    u0, v0 = EV[start_edge]

    ordered_edges = Int[start_edge]
    ordered_vertices = Int[u0, v0]
    used = Set([start_edge])

    current_v = v0
```

```
    while length(used) < length(edge_ids)
        candidates = [e for e in vedges[vpos[current_v]] if !(e in used)]
        if length(candidates) != 1
            return (ok=false,
                    reason="cannot continue unique loop traversal",
                    ordered_edges=ordered_edges,
                    ordered_vertices=ordered_vertices)
        end
        e = candidates[1]
        a, b = EV[e]
        next_v = (a == current_v) ? b : a
        push!(ordered_edges, e)
        push!(ordered_vertices, next_v)
        push!(used, e)
        current_v = next_v
    end

    if ordered_vertices[end] != ordered_vertices[1]
        return (ok=false,
                reason="component is not closed after traversal",
                ordered_edges=ordered_edges,
                ordered_vertices=ordered_vertices)
    end

    return (ok=true,
            reason="simple closed loop",
            ordered_edges=ordered_edges,
            ordered_vertices=ordered_vertices)
end

# ------------------------------------------------------------
# Local block extraction
# ------------------------------------------------------------

"""
    extract_blocks(W, Delta_0, Delta_1)

Extract contiguous local blocks from a block-diagonal dataset.

Conventions:
- `W` is d×nV or nV×d
- `Delta_0` is nE × nV   (edge-vertex)
- `Delta_1` is nF × nE   (face-edge)

Returns a vector of named tuples, one per original face block.
"""
function extract_blocks(W::AbstractMatrix,
                        Delta_0::SparseMatrixCSC,
                        Delta_1::SparseMatrixCSC)

    P = _normalize_points(W)   # d × nV

    nF, nE = size(Delta_1)
    nE0, nV = size(Delta_0)
    nE == nE0 || error("Incompatible sizes: Delta_1 has $nE edge-columns, Delta_0 has $nE0 edge-rows")

    face_edge_supports = [sort(_row_support(Delta_1, f)) for f in 1:nF]

    blocks = NamedTuple[]
    if nF == 0
        return blocks
    end

    fstart = 1
    current_edges = face_edge_supports[1]
    emin = minimum(current_edges)
    emax = maximum(current_edges)

    function finalize_block(flo, fhi, elo, ehi)
        local_edge_rows = elo:ehi
        used_vertices = sort(unique(v for e in local_edge_rows for v in _row_support(Delta_0, e)))
        vmap = Dict(v => k for (k, v) in enumerate(used_vertices))
        emap = Dict(e => k for (k, e) in enumerate(local_edge_rows))
        fmap = Dict(f => k for (k, f) in enumerate(flo:fhi))

        Wi = P[:, used_vertices]

        # local Delta0: local edges x local vertices
        I0 = Int[]
        J0 = Int[]
        V0 = Int8[]
        for e in local_edge_rows
            cols = _row_support(Delta_0, e)
            valsdict = Dict{Int,Int8}()
            _, cols_nz, vals_nz = findnz(Delta_0[e:e, :])
            for (c, val) in zip(cols_nz, vals_nz)
                valsdict[c] = Int8(val)
            end
```

```
            for c in cols
                if haskey(vmap, c)
                    push!(I0, emap[e])
                    push!(J0, vmap[c])
                    push!(V0, valsdict[c])
                end
            end
        end
        Delta0i = sparse(I0, J0, V0, length(local_edge_rows), length(used_vertices))

        # local Delta1: local faces x local edges
        I1 = Int[]
        J1 = Int[]
        V1 = Int8[]
        for f in flo:fhi
            cols = _row_support(Delta_1, f)
            valsdict = Dict{Int,Int8}()
            _, cols_nz, vals_nz = findnz(Delta_1[f:f, :])
            for (e, val) in zip(cols_nz, vals_nz)
                valsdict[e] = Int8(val)
            end
            for e in cols
                if haskey(emap, e)
                    push!(I1, fmap[f])
                    push!(J1, emap[e])
                    push!(V1, valsdict[e])
                end
            end
        end
        Delta1i = sparse(I1, J1, V1, fhi - flo + 1, length(local_edge_rows))

        return (
            id = length(blocks) + 1,
            W = Wi,
            Delta0 = Delta0i,
            Delta1 = Delta1i,
            v_offset = first(used_vertices),
            e_offset = elo,
            f_offset = flo,
            global_vertex_ids = used_vertices,
            global_edge_ids = collect(local_edge_rows),
            global_face_ids = collect(flo:fhi),
        )
    end

    for f in 2:nF
        supp = face_edge_supports[f]
        lo, hi = minimum(supp), maximum(supp)

        if lo <= emax + 1
            emin = min(emin, lo)
            emax = max(emax, hi)
        else
            push!(blocks, finalize_block(fstart, f - 1, emin, emax))
            fstart = f
            emin, emax = lo, hi
        end
    end
    push!(blocks, finalize_block(fstart, nF, emin, emax))

    return blocks
end

# ------------------------------------------------------------
# Boundary extraction per block
# ------------------------------------------------------------

"""
    block_boundary_edges(block)

Return local and global ids of boundary edges of the block.

Boundary edges are local edges used by exactly one local arranged face.
"""
function block_boundary_edges(block)
    Delta1 = block.Delta1
    nF, nE = size(Delta1)

    multiplicity = zeros(Int, nE)
    for e in 1:nE
        rows, _ = findnz(Delta1[:, e])
        multiplicity[e] = length(rows)
    end

    local_boundary_edges = findall(==(1), multiplicity)
    global_boundary_edges = block.global_edge_ids[local_boundary_edges]

    return (
```

```
        local_boundary_edges = local_boundary_edges,
        global_boundary_edges = global_boundary_edges,
        multiplicity = multiplicity,
    )
end

"""
    block_boundary_vertices(block)

Return local and global ids of vertices incident to boundary edges.
"""
function block_boundary_vertices(block)
    b = block_boundary_edges(block)
    Delta0 = block.Delta0

    verts = sort(unique(v for e in b.local_boundary_edges for v in _row_support(Delta0, e)))
    gverts = block.global_vertex_ids[verts]

    return (
        local_boundary_vertices = verts,
        global_boundary_vertices = gverts,
    )
end

"""
    block_boundary_loops(block)

Try to reconstruct the boundary loops of a block from its boundary edges.
"""
function block_boundary_loops(block)
    Delta0 = block.Delta0
    b = block_boundary_edges(block)

    EV = Vector{Vector{Int}}(undef, size(Delta0, 1))
    for e in 1:size(Delta0, 1)
        EV[e] = sort(_row_support(Delta0, e))
    end

    bedges = b.local_boundary_edges
    if isempty(bedges)
        return (
            ok = false,
            reason = "no boundary edges detected",
            components = Any[],
            loops = Any[],
        )
    end

    bverts = sort(unique(v for e in bedges for v in EV[e]))
    vpos = Dict(v => k for (k, v) in enumerate(bverts))
    adj = [Int[] for _ in bverts]
    for e in bedges
        u, v = EV[e]
        pu, pv = vpos[u], vpos[v]
        push!(adj[pu], pv)
        push!(adj[pv], pu)
    end
    for a in adj
        unique!(a)
        sort!(a)
    end

    comps_pos = _connected_components(adj)
    comp_vertices = [[bverts[k] for k in comp] for comp in comps_pos]

    loops = Any[]
    for comp in comp_vertices
        eset = [e for e in bedges if all(v -> v in comp, EV[e])]
        push!(loops, _loop_order_from_edges(EV, eset))
    end

    ok = all(getfield(L, :ok) for L in loops)

    return (
        ok = ok,
        reason = ok ? "all boundary components are simple closed loops" :
                      "some boundary component is not a simple closed loop",
        components = comp_vertices,
        loops = loops,
        boundary_edges = bedges,
    )
end

# ---------------------------------------------------------------
# Boundary validation
# ---------------------------------------------------------------

"""
```

```
    validate_block_boundary(block)

Validate the boundary support of one local face block.
"""
function validate_block_boundary(block)
    b_edges = block_boundary_edges(block)
    b_verts = block_boundary_vertices(block)
    loops = block_boundary_loops(block)

    odd_global_edges = Int[]
    bad_boundary_global_vertices = Int[]

    for (le, m) in enumerate(b_edges.multiplicity)
        if m == 0 || (m > 1 && isodd(m))
            push!(odd_global_edges, block.global_edge_ids[le])
        end
    end

    if !isempty(loops.boundary_edges)
        Delta0 = block.Delta0
        EV = [sort(_row_support(Delta0, e)) for e in 1:size(Delta0, 1)]
        bverts = b_verts.local_boundary_vertices
        deg = Dict(v => 0 for v in bverts)
        for e in loops.boundary_edges
            u, v = EV[e]
            deg[u] = get(deg, u, 0) + 1
            deg[v] = get(deg, v, 0) + 1
        end
        badloc = sort([v for (v, d) in deg if d != 2])
        bad_boundary_global_vertices = block.global_vertex_ids[badloc]
    end

    ok = loops.ok && isempty(odd_global_edges) && isempty(bad_boundary_global_vertices)

    return (
        ok = ok,
        block_id = block.id,
        boundary = loops,
        odd_global_edges = odd_global_edges,
        bad_boundary_global_vertices = bad_boundary_global_vertices,
        local_boundary_edges = b_edges.local_boundary_edges,
        global_boundary_edges = b_edges.global_boundary_edges,
        local_boundary_vertices = b_verts.local_boundary_vertices,
        global_boundary_vertices = b_verts.global_boundary_vertices,
        multiplicity = b_edges.multiplicity,
    )
end

# ------------------------------------------------------------
# Batch summaries
# ------------------------------------------------------------

"""
    summarize_block_boundaries(blocks)

Validate all blocks and return a summary.
"""
function summarize_block_boundaries(blocks)
    reports = [validate_block_boundary(b) for b in blocks]
    failed = [r.block_id for r in reports if !r.ok]

    return (
        ok = isempty(failed),
        nblocks = length(blocks),
        failed_blocks = failed,
        reports = reports,
    )
end

"""
    boundary_report(summary)

Compact textual report for all block boundary validations.
"""
function boundary_report(summary)
    lines = String[]
    push!(lines, "Block boundary summary")
    push!(lines, "  number of blocks      : $(summary.nblocks)")
    push!(lines, "  all blocks valid      : $(summary.ok)")
    push!(lines, "  failed blocks         : $(summary.failed_blocks)")

    for rep in summary.reports
        push!(lines, "")
        push!(lines, "  block $(rep.block_id): ok = $(rep.ok)")
        push!(lines, "    local boundary edges    : $(length(rep.local_boundary_edges))")
        push!(lines, "    local boundary vertices : $(length(rep.local_boundary_vertices))")
        push!(lines, "    odd/suspicious edges    : $(rep.odd_global_edges)")
        push!(lines, "    bad boundary vertices   : $(rep.bad boundary global vertices)")
```

```
        push!(lines, "    boundary loops ok       : $(rep.boundary.ok)")
        push!(lines, "    boundary reason         : $(rep.boundary.reason)")
    end

    return join(lines, "\n")
end

end # module
```

## Then use it like this

In a fresh Julia session:

```
include("BlockBoundaries.jl")
const BB = Main.BlockBoundaries

blocks = BB.extract_blocks(W, Delta_0, Delta_1)
summary = BB.summarize_block_boundaries(blocks)
println(BB.boundary_report(summary))
```

And to inspect one block:

```
rep7 = BB.validate_block_boundary(blocks[7])
rep7.ok
rep7.boundary
rep7.odd_global_edges
rep7.bad_boundary_global_vertices
```

When you run it, please paste the printed boundary summary.

---

Excellent. This is exactly the missing information.

You have **30 local face blocks**, and their sizes are now explicit. So we can stop guessing block boundaries and extract them by exact offsets.

## The three size arrays

From your Plasm output:

```
nverts = [18, 6, 18, 18, 6, 18, 18, 18, 17, 6, 17, 16, 6, 16, 17, 17, 19, 6, 19, 16, 6, 16, 19, 19, 10, 10, 10, 10

nedges = [25, 6, 25, 24, 6, 24, 25, 25, 23, 6, 25, 22, 6, 22, 24, 24, 27, 6, 27, 22, 6, 22, 26, 26, 10, 10, 10, 10

nfaces = [8, 1, 8, 7, 1, 7, 8, 8, 8, 1, 9, 7, 1, 7, 8, 8, 9, 1, 9, 7, 1, 7, 8, 8, 2, 2, 2, 2, 2, 2]
```

They are consistent with your bad dataset totals:

- sum(nverts) = `412`
- sum(nedges) = `534`
- sum(nfaces) = `159`

So these are exactly the offsets we need.

---

# What to change

The right fix is: **replace `extract_blocks(W, Delta_0, Delta_1)` with a version that uses `nverts`, `nedges`, `nfaces`.**

## Replace the current `extract_blocks` in `BlockBoundaries.jl` with this one

```
function extract_blocks(W::AbstractMatrix,
                        Delta_0::SparseMatrixCSC,
                        Delta_1::SparseMatrixCSC;
                        nverts::Vector{Int},
                        nedges::Vector{Int},
                        nfaces::Vector{Int})

    P = _normalize_points(W)   # d × nV

    sum(nverts) == size(P, 2) ||
        error("sum(nverts) = $(sum(nverts)) does not match number of vertices in W = $(size(P,2))")

    sum(nedges) == size(Delta_0, 1) ||
        error("sum(nedges) = $(sum(nedges)) does not match number of edge rows in Delta_0 = $(size(Delta_0,1))")
```

```
    sum(nverts) == size(Delta_0, 2) ||
        error("sum(nverts) = $(sum(nverts)) does not match number of vertex cols in Delta_0 = $(size(Delta_0,2))")

    sum(nfaces) == size(Delta_1, 1) ||
        error("sum(nfaces) = $(sum(nfaces)) does not match number of face rows in Delta_1 = $(size(Delta_1,1))")

    sum(nedges) == size(Delta_1, 2) ||
        error("sum(nedges) = $(sum(nedges)) does not match number of edge cols in Delta_1 = $(size(Delta_1,2))")

    nb = length(nverts)
    length(nedges) == nb || error("nverts and nedges must have same length")
    length(nfaces) == nb || error("nverts and nfaces must have same length")

    blocks = NamedTuple[]

    v0 = 1
    e0 = 1
    f0 = 1

    for bid in 1:nb
        nv = nverts[bid]
        ne = nedges[bid]
        nf = nfaces[bid]

        vrange = v0:(v0 + nv - 1)
        erange = e0:(e0 + ne - 1)
        frange = f0:(f0 + nf - 1)

        Wi = P[:, vrange]
        Delta0i = Delta_0[erange, vrange]
        Delta1i = Delta_1[frange, erange]

        push!(blocks, (
            id = bid,
            W = Matrix(Wi),
            Delta0 = SparseMatrixCSC(Delta0i),
            Delta1 = SparseMatrixCSC(Delta1i),
            v_offset = v0,
            e_offset = e0,
            f_offset = f0,
            global_vertex_ids = collect(vrange),
            global_edge_ids = collect(erange),
            global_face_ids = collect(frange),
        ))

        v0 += nv
        e0 += ne
        f0 += nf
    end

    return blocks
end
```

---

# How to call it now

In `test_bad1.jl`, define the three arrays before boundary analysis:

```
nverts = [18, 6, 18, 18, 6, 18, 18, 18, 17, 6, 17, 16, 6, 16, 17, 17, 19, 6, 19, 16, 6, 16, 19, 19, 10, 10, 10, 10

nedges = [25, 6, 25, 24, 6, 24, 25, 25, 23, 6, 25, 22, 6, 22, 24, 24, 27, 6, 27, 22, 6, 22, 26, 26, 10, 10, 10, 10

nfaces = [8, 1, 8, 7, 1, 7, 8, 8, 8, 1, 9, 7, 1, 7, 8, 8, 9, 1, 9, 7, 1, 7, 8, 8, 2, 2, 2, 2, 2, 2]
```

Then replace:

```
blocks = BB.extract_blocks(W, Delta_0, Delta_1)
```

with:

```
blocks = BB.extract_blocks(W, Delta_0, Delta_1;
    nverts=nverts,
    nedges=nedges,
    nfaces=nfaces)
```

---

# What should happen now

Instead of:

```
number of blocks : 1
```

you should get:

```
number of blocks : 30
```

and then the failing local blocks, if any, should start to appear.

That is the real test of your idea.

---

# One more small fix in `test_bad1.jl`

As I mentioned before, replace:

```
println("incident faces = ", [f for (k, face) in enumerate(res.FE) if e in face])
```

with:

```
println("incident faces = ", [k for (k, face) in enumerate(res.FE) if e in face])
```

---

# My expectation

Once block extraction is corrected, I expect the bad case to show:

- only a few failed local blocks,
- likely those corresponding to the mismatched independent arrangements,
- with suspicious boundary structure before global congruence.

That would validate your hypothesis very strongly.

When you run it, please paste the new `BB.boundary_report(summary)` output.

---

When you're ready, run the corrected version and paste the new boundary summary.

The two key edits are:

```
nverts = [18, 6, 18, 18, 6, 18, 18, 18, 17, 6, 17, 16, 6, 16, 17, 17, 19, 6, 19, 16, 6, 16, 19, 19, 10, 10, 10, 10
```

```
nedges = [25, 6, 25, 24, 6, 24, 25, 25, 23, 6, 25, 22, 6, 22, 24, 24, 27, 6, 27, 22, 6, 22, 26, 26, 10, 10, 10, 10
```

```
nfaces = [8, 1, 8, 7, 1, 7, 8, 8, 8, 1, 9, 7, 1, 7, 8, 8, 9, 1, 9, 7, 1, 7, 8, 8, 2, 2, 2, 2, 2, 2]
```

and:

```
blocks = BB.extract_blocks(W, Delta_0, Delta_1;
    nverts=nverts,
    nedges=nedges,
    nfaces=nfaces)
```

Also fix the diagnostic line to:

```
println("incident faces = ", [k for (k, face) in enumerate(res.FE) if e in face])
```

Then run:

```
include("test_bad1.jl")
```

What I most want to see is:

```
println(BB.boundary_report(summary))
```

especially whether it now says `number of blocks : 30` and which block IDs fail.

---

---

This is a very important result.

It tells us that your intuition was **exactly right**, but with one refinement.

## What we have learned

### 1. The bad example is really bad globally

The global 2-skeleton fails at balance stage, with odd edge-face multiplicities on:

```
[36, 38, 41, 66, 68, 74, 96, 98, 99, 114, 178]
```

So the global `EF` cannot be oriented because some global edges have incidence count **1** or **3**, instead of an even number.

---

### 2. But every local block boundary is individually valid

This is the decisive point.

Your block-boundary analysis says:

- `number of blocks = 30`
- `all blocks valid = true`

So:

- each independent face arrangement is internally coherent,
- each local boundary decomposes into valid closed loops,
- there are no local boundary defects detectable **inside a single block**.

This is a strong result.

---

## Therefore the defect is not:

- **inside one block alone**

but rather:

- in the **boundary congruence between different blocks**

That is now the real target.

And that matches very well what you had predicted earlier.

---

# The most revealing evidence

Look at the odd global edges:

```
edge 98  => [37, 73]   incident faces = [113]
edge 99  => [37, 75]   incident faces = [49]
edge 178 => [73, 75]   incident faces = [63]
```

These three are the strongest smoking guns.

They each have only **one** incident face in global `FE`.

So the triangle

- `[37,73]`
- `[37,75]`
- `[73,75]`

appears as an **unmatched small loop**.

This is exactly the kind of artifact you described a few days ago:

> a couple of nodes formed a single empty loop with three edges

That is almost certainly what happened here.

---

## The other odd edges have multiplicity 3

These ones:

- 36 = `[13,14]`
- 38 = `[13,37]`
- 41 = `[14,38]`
- 66 = `[25,27]`
- 68 = `[25,45]`
- 74 = `[27,46]`

- 96 = `[37,38]`
- 114 = `[45,46]`

have **three** incident faces.

So they are not missing alone; they are part of a mismatch region where one extra local face contribution was created or not matched away.

This again points to **inter-block mismatch**, not to a single bad local boundary loop.

---

# Main conclusion

Your present test proves:

## Local block topology is fine.

## Global failure is caused by incoherent matching of block boundaries across adjacent original faces.

So your next task is no longer:

- validate each block boundary support by itself

but:

- compare the boundary supports of **adjacent blocks** and verify they induce the same subdivision on their common geometric support.

That is now the correct target.

---

# What the odd edges suggest geometrically

The set

```
[37,73], [37,75], [73,75]
```

looks like a little triangular cycle that survived only on one side.

That means one block probably produced a subdivision feature that its adjacent block did not reproduce on the same shared support.

So the next algorithm should detect things like:

- one block has boundary segment split at a point,
- the adjacent block keeps the same segment unsplit,
- or one block introduces a tiny extra boundary loop,
- or one side contains a subdivision vertex absent on the other side.

---

# What to do next

We now need a new routine:

## pairwise boundary congruence of blocks

For each pair of blocks that should meet along a common support:

1. extract their boundary vertices and boundary edges;
2. map their geometry into a common comparison space;
3. detect coincident boundary supports;
4. verify that the induced 1D decompositions are identical;
5. report:

- missing subdivision vertices,
- unmatched split edges,
- extra tiny loops.

That is the real next step.

---

# A very useful immediate observation

The three singleton edges

```
98  => [37,73]
99  => [37,75]
178 => [73,75]
```

already identify one suspect local region.

So the next pairwise checker should probably start from the blocks that contain the faces:

```
49, 63, 113
```

because those are the only incident faces of the singleton edges.

That gives us a first localized defect zone.

---

# Small side note

This warning:

```
Assignment to `rep` in soft scope is ambiguous
```

is not mathematically important. It is only a Julia scope warning and can be ignored for now.

---

# Final interpretation

This run is a success.

It has shown that:

- your bad example is correctly detected,
- the failure is not due to gross local invalidity,
- and the true problem is **boundary mismatch between independently computed adjacent face arrangements**.

That is a very strong and useful diagnosis.

Tomorrow, or next, I would recommend that we write the next module:

```
validate_block_pair_boundaries(...)
```

starting from the suspect region around global edges:

```
[98, 99, 178]
```

and their incident faces:

```
[49, 63, 113]
```